

StarfishOS: Revisiting Single System Image on CXL with State-Partitioned Microkernel

Submission #906

Abstract

Single System Image (SSI) offers an attractive OS abstraction that enables single-machine applications to transparently execute on multiple machines with improved resource utilization. The emergence of Compute Express Link (CXL) shifts cross-machine synchronization from message passing to coherent shared memory, which may help overcome SSI's long-standing adoption barrier caused by inefficient cross-machine message passing. However, placing state in slow CXL memory degrades performance and makes the system vulnerable to partial failures, since inconsistent CXL-resident state may block or crash the whole system.

This paper notes that microkernels' minimized kernel-space state and capability-based state management allow easier state partitioning to minimize state on CXL. To this end, we propose a *state-partitioned microkernel*, and realize it in Starfish, the first OS-level SSI that leverages CXL shared memory for both high efficiency and partial failure resilience by keeping CXL state minimal and recoverable. Evaluations show that applications can transparently scale on StarfishOS with zero code change, and achieve up to 4.2× and 4.1× the throughput of their distributed and CXL-optimized versions, respectively. Moreover, StarfishOS utilizes all stranded CPU resources by running applications across machines.

1 Introduction

Compute Express Link (CXL) [3] connects rack-scale machines into a *CXL pod* [64], where all machines access a shared memory pool through load/store [4]. This shared-memory primitive replaces inter-machine messaging with load/store access, enabling much cheaper cross-machine synchronization. Prior work has already demonstrated the potential of this model. For example, CXL-SHM [70] builds a MapReduce program from scratch to exchange shuffle data through CXL shared memory to reduce execution time. Tigon [37], on the other hand, reorganizes database data structures to be CXL-aware, avoiding two-phase commit [32] and improving OLTP throughput by 18.5×.

Despite their performance gains, a broader question arises: to benefit from CXL, is it necessary to re-design and re-implement every application, especially those with large and hard-to-modify codebases?

To answer this question, we revisit OS-level single system image (SSI) [26, 29, 58, 63] as a way to enable existing single-machine shared-memory applications to transparently benefit from CXL. SSI exposes a CXL pod as one logical machine with a single address space, PID namespace, file namespace, and device namespace. Under SSI, the OS

transparently schedules application threads pod-wide and routes all cross-machine operations through CXL load/store, without any application redesign.

Building SSI on CXL, however, is non-trivial. At one extreme, the system can keep all OS and application state in local DRAM and communicate across machines via message passing, as traditional SSI systems do [26, 29, 63]. However, this design fails to exploit CXL's shared memory semantics and remains inefficient due to message-passing [24, 42, 48, 59]. At the other extreme, the system can place all OS and application state on CXL, so the pod behaves like a shared-memory machine on which existing Oses and applications run unmodified. While simple and fully exploiting CXL's shared memory semantics, this design faces two obstacles:

(1) *Efficiency*: CXL memory is slower than local DRAM: 2–5× access latency and 20%–80% bandwidth of local DRAM [55, 60, 70]. Placing all kernel and application state in CXL causes unacceptable overhead for kernel and applications (up to 3.4× slowdown on our platform, as shown in Figure 13).

(2) *Partial failure*: in CXL pod, host machines and CXL memory modules have separate power supplies and can fail independently [70, 74]. A host crash may leave CXL-resident locks unreleased or leave data structures in inconsistent states; surviving machines that observe them may block or crash, turning a single-host failure into a pod-wide outage.

Key insight. Both obstacles stem from the same cause: placing too much OS state on CXL. *State partitioning* resolves both simultaneously, i.e., keeping only the minimal coordination state in CXL, and leaving everything else in private DRAM for efficiency. Because this shared state is small, it can be engineered for partial-failure resilience, so a host crash need not block or crash the surviving machines.

Realizing this in practice requires a *clear state-management abstraction*: the OS must explicitly identify each state object, its type, and its dependency edges. Monolithic kernels generally lack such an explicit model—their state is spread across tightly coupled kernel subsystems with implicit references—so fine-grained partitioning and placement become impractical [36, 57]. Microkernels, in contrast, are suited for state partitioning. First, their kernel-space state is small, enumerable, and individually manageable [39, 69]. Second, microkernels' capability-based state management tracks each OS object as a first-class entity with explicit type and capabilities for access control, enabling per-object placement decisions [39, 69].

Contributions. We present StarfishOS, the first OS-level SSI for CXL pods with a *state-partitioned microkernel*. Each layer of the OS stack, including kernel-space modules (IPC,

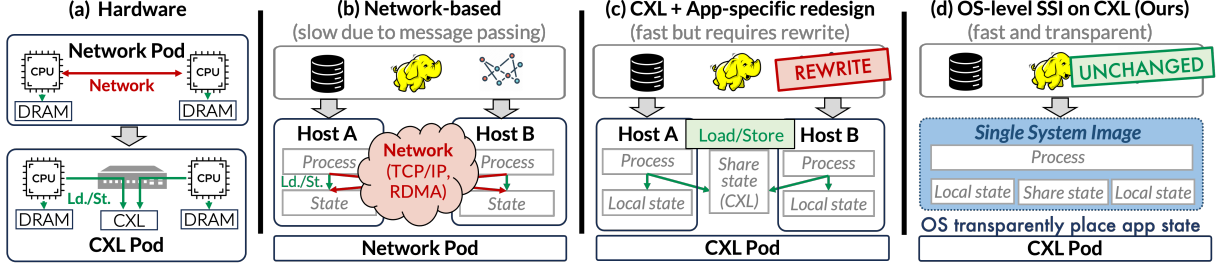


Figure 1. Compare network-based and app-specific redesign approaches with our OS-level SSI for CXL Pod.

scheduling, memory management), system services (filesystems, process managers, etc.), and applications, faces a distinct state-sharing challenge; StarfishOS addresses each with a tailored partitioning and OS-level resilience strategy.

For efficiency, StarfishOS applies the same partitioning rule across the OS stack: only state needed for cross-machine coordination resides in CXL, while locality-sensitive state remains in private DRAM. In the kernel, this means sharing only the data structures that implement inter-machine functionality, such as IPC queues, scheduling queues, and the CXL allocator. In system services, StarfishOS designs collaborative system service instances to keep only global coordination metadata in CXL and leave service state inside per-machine service instances. For applications, StarfishOS uses OS-visible semantics to partition application state into objects and place each object in either CXL or private DRAM. StarfishOS also introduces a page-fault-driven mechanism that migrates heap pages to CXL only on demand. This makes CXL usage small and aligned with actual state sharing rather than with putting everything on CXL.

For handling partial failures, StarfishOS targets OS resilience rather than transparent application recovery. A machine crash should therefore resemble the loss of processes on a single machine, without blocking or corrupting the rest of the pod. To achieve this, StarfishOS makes its CXL-resident kernel coordination state recoverable with lock-free, detectably recoverable data structures [18, 31], keeps most service state outside CXL so failed service instances can be re-created via micro-reboot, and leaves application-level recovery to the applications' own mechanisms. As a result, surviving machines can continue using both the OS and the shared CXL state after other machines fail.

Evaluation. We evaluate StarfishOS's two main benefits for existing single-machine applications. First, StarfishOS transparently scales unmodified shared-memory applications to the full pod: applications such as Phoenix [50], DBx1000 [5], and GeminiGraph [73] scale to 8 machines with 2.0–3.5× speedup and zero code changes. For Phoenix and GeminiGraph, StarfishOS outperforms their distributed-style baselines by 1.7× and 4.2×, respectively. For DBx1000, StarfishOS achieves 4.1× speedup compared to Tigon [37]. Second, StarfishOS enables applications to utilize all stranded CPU resources by running applications across machines. In

traditional OSes, a multithreaded application must be bin-packed onto one machine, so fragmented CPUs across machines cannot be used. In contrast, StarfishOS lets applications use all idle CPUs across machines.

In summary, this paper makes the following contributions:

- We revisit SSI in the CXL era and present the first OS-level SSI that uses CXL's shared-memory primitive for cross-machine synchronization.
- We propose the state-partitioned microkernel to achieve efficient SSI with OS-level partial-failure resilience.
- We support automatic scaling of existing single-machine applications across multiple machines and improve pod-wide resource utilization.

2 Revisiting Single System Image with CXL

CXL can yield application performance gains. Compute Express Link (CXL) [3] enables multiple hosts to access a shared memory pool using load and store instructions. In a CXL pod [64], this hardware-supported shared memory can reduce the coordination overheads of message-passing (Figure 1). Recent studies provide evidence for such benefits in various workloads. For databases, Tigon [37] places cross-partition data in CXL and coordinates concurrent accesses through shared-memory primitives, thereby avoiding the costly two-phase commit (2PC) [32] for distributed transactions. Tigon reports up to 18.5× TPCC [12] throughput improvement. For MapReduce workloads, CXL-SHM [70] places shuffle data in shared CXL memory so that workers on different machines can access it directly, reducing communication overhead and improving the execution time of applications like WordCount and KMeans.

Current gains often require application-specific redesign. So far, the large performance gains often rely on *application-specific redesign*. For example, Tigon [37] introduces more than 5,000 lines of changes to an 18,000-line codebase to restructure database data structures, and manage data placement between local DRAM and shared CXL memory. Despite this effort, Tigon remains specialized to locking-based concurrency control, including 2PL [32] and next-key locking [32]. CXL-SHM [70] exhibits a similar pattern: although it reports comparable gains, it rewrites MapReduce from scratch to accommodate its pass-by-reference design.

Our choice: OS-level SSI. We therefore ask whether applications can gain CXL’s benefits *without per-application redesign*, and whether a single approach can apply to a *broad class* of applications. Since decades of experience show that single-machine, multithreaded applications already exploit shared-memory semantics effectively, we explore a different path: an *OS-level single system image (SSI)* that makes a CXL pod appear as a large shared-memory machine. SSI brings two direct advantages:

(1) *Auto-scaling existing programs to transparently benefit from CXL*: legacy single-machine applications, whether single- or multi-process, can be auto-scaled to run on any CPU in the CXL pod with zero code changes. As shown in Figure 1 (d), the OS transparently schedules a process’s threads across the pod and places its state on DRAM/CXL memory on demand, so cross-node communication transparently becomes load/store rather than message passing.

(2) *Better resource utilization and failure handling*: with a single view of pod-wide resources provided by SSI, applications can transparently use remote devices such as SSDs/NICs. Prior work reports that pooling devices across 8 hosts can increase device utilization by 46–81% [72]. Applications can fail over to remote devices when local ones fail, avoiding repair downtime [71, 72].

SSI is appealing in cloud environments. A tenant may submit a legacy in-memory analytics job written for single-machine, but the provider may have only fragmented resources across a CXL pod rather than one sufficiently large machine. Without SSI, the tenant must wait for a sufficiently large server or rewrite the application for distributed or CXL-aware execution. With OS-level SSI, the provider can aggregate pod-wide resources and run the job transparently as one larger machine, without application changes.

Why not choose user-level or hypervisor-level SSI? We consider three possible layers for realizing SSI on a CXL pod: (1) user-level SSI, i.e., distributed shared memory (DSM) libraries [17, 24, 42, 44, 48, 59], (2) hypervisor-level SSI [27, 52, 56], and (3) OS-level SSI [19, 26, 29, 45, 58, 63]. We choose OS-level SSI because it is the only approach that can *both* transparently run existing shared-memory applications and globally manage pod-wide resources.

User-level SSI, i.e., DSM libraries [17, 24, 42, 44, 48, 59], also lets programmers write as if on a single machine, but CXL-based DSMs still require *application-specific redesign* for efficient DRAM/CXL data placement. They also cannot provide pod-wide resource pooling that we target.

Hypervisor-level SSI [27, 52, 56] uses a distributed hypervisor to aggregate rack-scale resources. However, enabling applications to run correctly and efficiently on a CXL pod still requires substantial guest-OS changes, including scheduling, memory placement, and system services. For SSI, implementing the system at the OS level is therefore more effective

than doing so at the hypervisor level, because the OS has richer semantic visibility into OS components.

3 State-Partitioned Microkernel

3.1 Challenges of building SSI atop CXL

Building OS-level SSI atop CXL requires both efficiency and robustness: it should use shared memory for cross-machine synchronization while *partitioning system state* to minimize the amount of state placed in CXL, and it should tolerate partial machine failures so that partially updated shared state does not block or crash surviving machines.

3.2 Why microkernels are a good fit

We find microkernel architecture a good fit for building SSI atop CXL for two main reasons:

(1) **Decoupled and Structured State Organization**: A microkernel gives system state a clearer organization than a monolithic kernel. At a coarse granularity, it *decouples* kernel state from service state: the kernel retains only a small amount of state for basic mechanisms (e.g., IPC and scheduling), while most OS services, including file systems and device drivers, run in user space with their own state. As a result, major categories of system state are placed behind explicit boundaries, rather than being mixed inside the kernel space like in monolithic kernels.

At a finer granularity, capability-based microkernels organize state through capabilities [34] in a *structured* way, which represents system state as objects with explicit relations among them [39, 54, 66, 69]. This gives system state a finer-grained and structured organization, making dependencies easier to reason about. By contrast, although monolithic kernels also maintain kernel objects, these objects are often spread across in the kernel-space and connected through implicit dependencies. Consequently, system state is harder to isolate and partition correctly.

(2) **Easier service recovery**: microkernels can recover failed user-space system services by micro-rebooting them [25, 46], i.e., restarting only the failed service instance without rebooting the whole OS. In contrast, in monolithic kernels, filesystems and many other services run inside the kernel and share kernel state with the rest of the system, making isolated restart much harder. Micro-rebooting is especially useful under partial failures, because if one machine fails and takes the service down with it, the service can be restarted on another machine without rebooting the whole OS.

3.3 State-partitioned microkernel

Based on microkernel’s advantages of decoupled and structured state organization and easier service recovery, we propose a *state-partitioned microkernel architecture* for SSI atop CXL. It explicitly divides SSI state into three layers: (1) kernel-space modules, (2) user-space system services, and (3) applications. This structure lets us use layer-specific

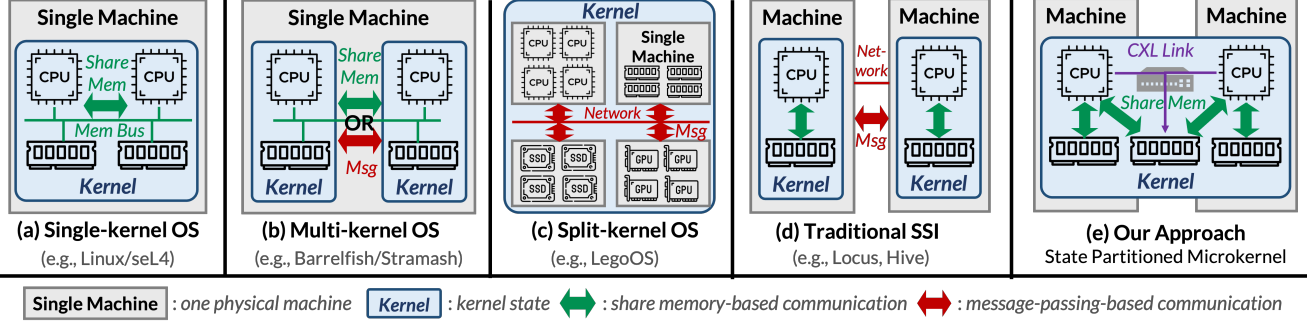


Figure 2. Kernel architecture of existing OSes and our proposed state-partitioned microkernel.

state-partitioning and partial-failure-handling mechanisms as summarized in Table 1.

3.4 Differences with Existing OSes

As shown in Figure 2, our proposed state-partitioned microkernel departs from existing OSes in three main aspects:

(1) *Cross-machine coordination with efficient shared memory instead of message passing.* Single-machine OSes (Figure 2(a)) assume only local memory and do not support cross-machine coordination. Multi-kernel OSes [19, 20] (Figure 2(b)), SSI OSes [26, 58, 63] (Figure 2(d)) and disaggregated OSes such as LegoOS [53] (Figure 2(c)) use multiple kernel instances and coordinate them via message passing. In contrast, StarfishOS directly constructs kernel-space data structures in shared memory for cross-machine coordination.

(2) *A unique partial-failure model.* Existing systems generally assume a single failure domain (for single-machine OSes, Figure 2(a) and (b)), or assume that when a machine fails, its CPU execution and OS state fail together (for SSIs and disaggregated OSes, Figure 2(d) and (c)). They all fail to handle the partial failure model introduced by CXL, where a machine’s CPU and memory may fail while its kernel and application state remains accessible in CXL. In contrast, StarfishOS is designed explicitly for this partial-failure model.

(3) *Extending resource disaggregation to CPUs.* Prior disaggregated systems mainly separate memory and storage while CPU execution remains organized per machine. For example, LegoOS [53] splits resources into CPU, memory, and SSD nodes, allowing multiple CPU nodes to share a common memory/storage pool, but each process still runs on a single CPU node. In contrast, StarfishOS extends disaggregation beyond memory and storage to CPUs across machines.

4 Design Overview

4.1 StarfishOS in Action: A MapReduce Case

We use a simple MapReduce program to illustrate the core capabilities and components required by StarfishOS. Running MapReduce across a CXL pod requires:

- *Step 1: Create a thread pool and place mapper/reducer threads across machines.* This requires StarfishOS’s **scheduler** to schedule threads on each CPU core in the pod.

- *Step 2: Initial threads load input data from the filesystem.* This requires (1) StarfishOS’s **filesystem services** to provide a global filesystem namespace so that threads across the CXL pod can read the input file regardless of where it is physically stored, and (2) **cross-machine IPC** to invoke the filesystem service.
- *Step 3: Mappers compute data and place intermediate results into shared memory buffers.* This requires StarfishOS’s **memory manager** to enable allocation of shared memory buffers from DRAM or CXL memory.
- *Step 4: After mappers complete, use conditional variables to wake up reducers.* This requires a **notification mechanism** that supports waking up threads on other machines.

4.2 Basic Components of StarfishOS

As shown in Figure 3, StarfishOS consists of the following components to support the above workflow:

(1) *Bootloaders.* Each machine has a bootloader responsible for gathering local resources. Bootloaders coordinate with each other to boot StarfishOS, and they build a unified kernel virtual address space, so that machines across the pod execute in a consistent address layout.

(2) *Unified kernel-space modules (§5)* span multiple machines to provide core OS mechanisms, including IPC (*Step 2*), scheduling (*Step 1*), notification (*Step 4*), and memory management (*Step 3*). Together, these modules provide a unified kernel abstraction across CXL pod, and their state is distributed across per-machine DRAM and CXL.

(3) *Collaborative system services (§6)* present a single global view across machines—a single PID space, filesystem namespace, and device namespace—so that applications can seamlessly use both local and remote services (for *Step 2*). We design multiple service instances running on different machines that operate collaboratively, keeping instance’s state in private memory while only coordination metadata in CXL.

(4) *Applications (§7)*, with the above three components, can transparently run across the CXL pod.

4.3 Hardware, Failure Model, Base Microkernel

Hardware setup. We target a CXL pod hardware setup [64] (Figure 2 (e)), where rack-scale machines are equipped with

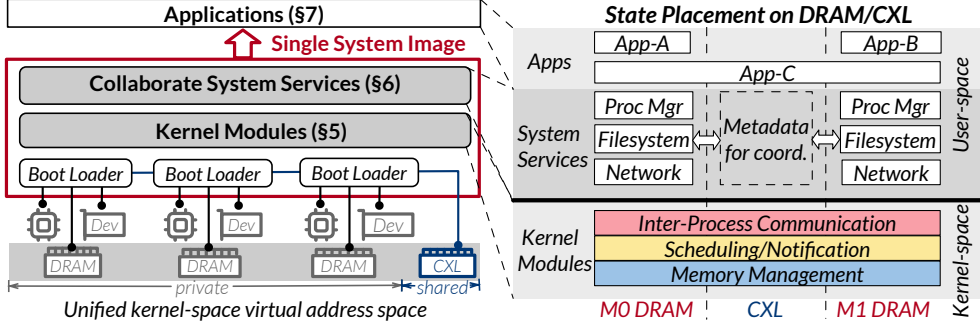


Figure 3. StarfishOS architecture. StarfishOS is an OS-level SSI with unified kernel-space modules (§5), collaborative user-space system services (§6), and bootloaders (§4). With StarfishOS, applications (§7) run on a CXL pod as if they were running on a single machine.

Table 1. State partition and partial failure handling of each component in StarfishOS.

Component	Runtime shared state	Partial failure handling
K-space func.	IPC (§5.1)	Detectably recoverable queues
	Notification (§5.3)	Detectably recoverable queues; libc timeouts mitigate higher-level primitives
	Scheduler (§5.2)	Detectably recoverable queues
	Mem manager (§5.4)	Lock-free buddy system & light undo logs
U-space	Sys services (§6)	Micro-reboot service instances on surviving machines and repair state gaps.
	Applications (§7)	Applications recover with their own mechanisms; the OS remains usable on surviving machines and reclaims resources from failed ones.

private memory (i.e., local DRAM) and *shared memory* (i.e., memory connected through the CXL switch [3]).

We assume hardware cache coherence for CXL, but use only a small coherent region (several hundred MiBs). Although prior work [37] and vendors [38] suggest supporting coherence only for a small region, this mainly reflects hardware-complexity tradeoffs rather than a limitation: both Arm’s discussion [47] and FPGA prototypes [61] indicate that broader coherence is feasible. Still, StarfishOS minimizes coherent-memory usage to reduce coherence overhead [61].

We assume CPU caches are lost upon machine failures, and rely on cacheline flushes for partial-failure handling. Hardware support such as global persistent flush [51] could eliminate cacheline flushes and further improve performance.

Failure model. StarfishOS tolerates partial failures [70]. It assumes that CXL is always reliable and available as in prior works [67, 70], while physical machines may fail independently. It assumes fail-stop failures of entire physical machines (including CPU and DRAM), but not fail-slow or Byzantine failures. It also assume reliable machine failure detection using heartbeat method proposed in prior work [74].

Behavior under partial failures. Under failures of one or several machines, StarfishOS preserves OS functionality on surviving machines, but does not provide transparent application recovery. (1) Preserving OS functionality means that all kernel-space mechanisms remain available on surviving machines, and system services become usable again after recovery. (2) No transparent application recovery

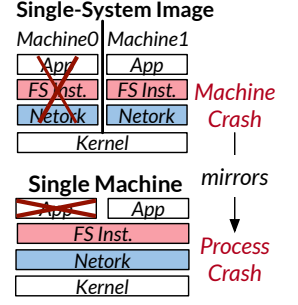


Figure 4. Failure model: a machine crash in SSI behaves like the crash of processes in single-machine OS.

means that applications running on failed machines terminate abruptly, analogous to process crashes in a conventional single-machine OS, while applications on surviving machines continue unaffected (as shown in Figure 4).

Base microkernel and our modifications. StarfishOS is built on ChCore [13, 33, 66], which is a capability-based microkernel like seL4 [39]. ChCore provides IPC, scheduler/notification, and memory management in kernel space, and system services such as process management and filesystems in user space. ChCore uses musl-libc [8] for POSIX compliance, and can run various applications (including Redis [9], Memcached [7], RocksDB [10], etc.). Although our prototype is built on a specific microkernel, the techniques described here also apply to other microkernels.

5 Kernel-Space Modules

This section describes how we realize SSI by extending kernel-space modules to support cross-machine functionalities.

Our *core design principle* is to partition each kernel module’s state into DRAM and CXL data structures for local and remote mechanisms (§5.1–§5.4), and make CXL data structures partial-failure resilient using lock-free, crash-consistent data structures [18, 31] (§5.5). Table 1 summarizes our used methods for each component.

5.1 Inter-Process Communication

IPC is the microkernel’s mechanism for invoking system services. In our base kernel, IPC follows the L4/seL4 style [1, 21]: it is a function call-like, user-controlled context switch to a

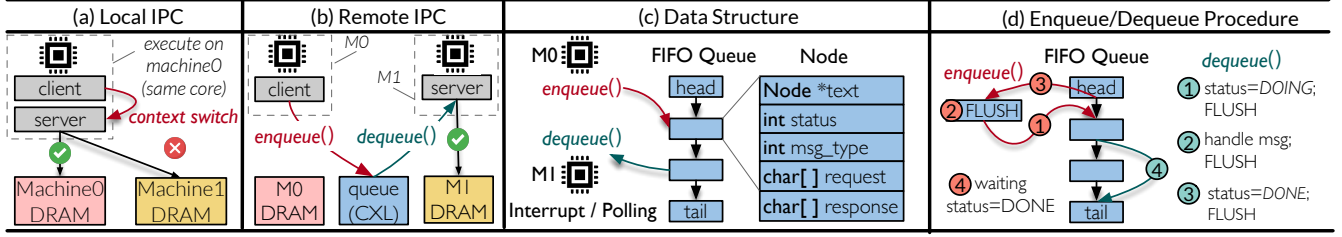


Figure 5. Inter-Process Communication.

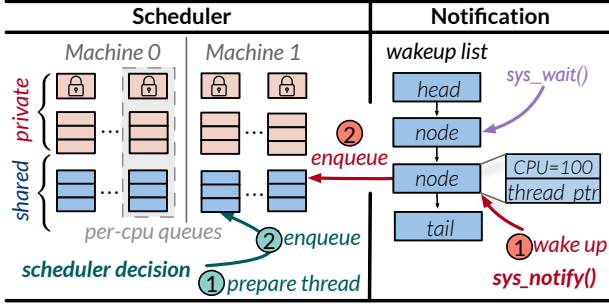


Figure 6. Scheduler/Notification.

server thread without scheduler involvement. This design works for local IPC within a machine, but it does not extend naturally to cross-machine remote IPC when the server’s address space and state remain in remote DRAM. We therefore keep the original design for local IPC, and implement cross-machine IPC as message passing over CXL.

Local IPC does not extend across machines (Figure 5 (a)). In our base kernel, local IPC follows the L4/seL4 style: the client transfers control directly to the server thread, which then executes in the server’s address space. For a remote server, however, its address space and state remain on the remote machine, so switching control alone is not sufficient. A straightforward alternative is to place server state in CXL so that multiple machines can directly operate on it. We do not choose this design because: (1) it would consume a large amount of scarce cache-coherent CXL memory; (2) under partial failure, a machine crash may expose partially updated server’s state to other machines, making partial failures harder to recover from.

Remote IPC (Figure 5 (b)). We therefore adopt a CXL-based message-passing design for cross-machine IPC. As shown in Figure 5 (b), each service provides a dedicated handler thread for cross-machine IPC, separate from its local IPC. Each such handler thread owns a FIFO request queue in CXL for cross-machine IPC requests. The client creates a message node on CXL (Node), prepares the request, enqueues it to the target machine’s queue, and, if needed, waits for `status==DONE`; the server dequeues the node, handles the request, writes the response, and sets the status to `DONE`.

5.2 Scheduling

StarfishOS extends thread scheduling beyond a single machine: it can assign a thread to any processing core in the

CXL pod, including remote cores. Once assigned, the scheduler on the target core treats the thread as a local schedulable entity and schedules it according to the scheduling policy.

Local scheduling. As shown in Figure 6, the base microkernel uses per-core scheduling queues (*private queue*) that store runnable *thread control blocks*. A thread control block contains the execution state needed to resume a thread (e.g., saved registers), and the scheduling state (e.g., priority, affinity). Scheduler on each core selects a runnable thread from its private queue and context-switches to it.

Remote scheduling. To support scheduling a thread on remote machines, StarfishOS equips each core with an additional *shared queue* in CXL (Figure 6). Threads assigned to a remote core are enqueued into shared queue, allowing the target core to admit them into its local queue for scheduling. The shared queue is checked along the target core’s normal scheduling path, thus avoiding the overhead of sending explicit cross-machine scheduling requests. Note that a thread scheduled to a remote machine must already reside either in that machine’s private memory (for notifications, §5.3) or in CXL (for cross-machine application execution, §7), so that the remote machine can access it.

5.3 Notification

Notification is the kernel mechanism for event-driven wakeup and serves as the basis for higher-level synchronization primitives such as timeouts, futures, and semaphores. StarfishOS extends notification to support cross-machine wakeup.

Local notification. As shown in Figure 6, our base microkernel maintains a *waiting-list* of threads waiting on a notification. To wait for an event, an application thread invokes `sys_wait` to insert its *thread control block* into the waiting-list. When the event occurs (i.e., `sys_notify` is invoked or a timeout fires), the kernel wakes one waiting thread by dequeuing it and scheduling it.

Remote notification. StarfishOS supports cross-machine notification by placing the notification’s waiting-list in CXL. Thus, all machines can invoke `sys_wait/sys_notify` that operate on the shared waiting-list. However, the waking machine may still be unable to directly access the target thread’s control block in another machine’s private DRAM. To address this, each waiting-list node becomes a small *thread node*: it stores a pointer to the target thread together with its `cpu_id`, allowing the waking machine to enqueue the

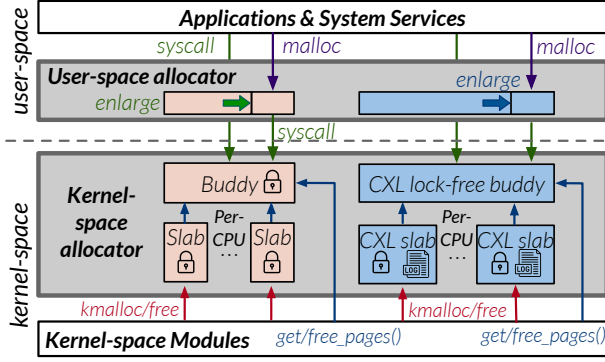


Figure 7. Memory Manager.

wakeup to the correct target CPU without directly dereferencing remote private thread state.

5.4 Memory Management

Memory management in StarfishOS supports both machine-local DRAM and pod-wide CXL memory.

Original memory management. As shown in the left part of Figure 7, similar to conventional kernels, the base microkernel uses a buddy/slab organization [22, 40] in kernel space and a libc allocator in user space. The kernel allocator serves kernel-space module’s allocations, while the user-space allocator obtains memory from the kernel and serves user-space processes’ allocations.

StarfishOS’s memory management. StarfishOS extends memory management to support both machine-local DRAM and pod-wide CXL memory. It uses a separate DRAM allocator on each machine and a shared CXL allocator for the entire pod. In kernel space, StarfishOS retains the conventional buddy/slab structure, but replaces the CXL buddy allocator with a lock-free buddy system [65] for partial failure resilience (discussed later in §5.5). The slab allocator remains per-CPU: allocation is served by the local slab instance, while deallocation is routed back to the owning machine for reclamation. In user space, StarfishOS extends the libc allocator to manage both DRAM and CXL memory; applications can choose the target memory type, and the default is the DRAM of the machine on which the application runs.

5.5 Partial Failure Handling

Since StarfishOS places shared kernel state on CXL, it must ensure that partial failures neither corrupt shared state nor block surviving machines. For example, if a machine fails while holding a lock in CXL, the lock will not be released and may block other machines.

To avoid this, our *design principle* is to use lock-free and crash-consistent data structures for shared kernel state [18, 31]. Lock-freedom prevents failed machines from blocking others, while crash-consistency ensures that crashes do not leave shared state corrupted and that recovery can handle in-flight operations [18, 31].

We defer details of our implementation to supplementary material due to space constraints.

Durable Queue for IPC, Scheduler, Notification. As listed in Table 1, we implement the IPC queue, scheduler queue, and notification waiting-list with DurableQueue, which is a lock-free and detectably recoverable FIFO queue proposed by Friedman et al. [31]. As shown in Figure 5 (d), enqueue and dequeue linearize with a single CAS, so a partial failure never leaves the queue itself corrupted or permanently blocked. Building on this property, StarfishOS handles failures differently depending on whether they occur at the enqueuer or dequeuer side:

- *Enqueuer Crash:* If the enqueuer’s machine fails during enqueue, the queue remains usable because the operation is lock-free. Other machines can still enqueue IPC messages, schedule threads, or wait for events.
- *Dequeuer Crash:* If the dequeuer’s machine fails during dequeue, the operation may be lost or partially executed, but it does not block other machines. Recovery is component-specific: IPC returns ABORT or relies on service-level retry; scheduler reconstructs local state and retries; notification returns ABORT, allowing libc to report timeouts.

CXL allocator. StarfishOS maintains kernel-space buddy/slab allocator and user-space libc allocator for CXL memory. We ensure their partial failure resilience as follows:

- *Lock-free Buddy:* We choose LLFree [65], a lock-free and crash-consistent allocator, to implement CXL buddy system. Under partial failure, the buddy state remains consistent, and other machines can still allocate/free memory without blocking since LLFree is lock-free.
- *Per-CPU Slab with Undo-log:* Each CPU has its own slab instance, so allocation on slab stays local and does not block allocation on other machines. Free operations on slab allocator are forwarded asynchronously to the owner machine. To safely reuse slab instances after recovery, StarfishOS keeps a lightweight in-memory undo-log to undo in-flight slab operations.
- *Libc allocator:* each application uses an independent libc allocator instance. Thus, the failure of libc allocator will not block other applications.
- *Memory Leak Prevention:* Allocator metadata remains consistent under partial failure, but memory may be allocated before it is linked to OS (e.g., a page allocated but not yet mapped into a process). StarfishOS prevents such leaks by using microkernel’s capability-based state organization (§7.1) as the ground truth and checking allocator records against it during recovery.

6 Collaborative System Services

6.1 State Partition with Collaborative Instances

System services in StarfishOS are distributed as collaborative user-space instances. This keeps most service state in private

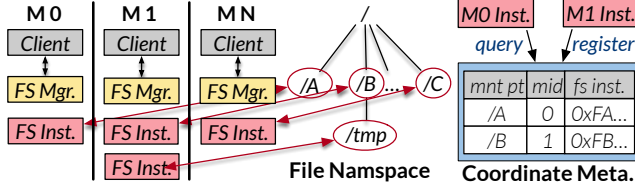


Figure 8. Collaborative filesystem service. Filesystem instances are distributed across machines, each owns a disjoint part of file namespace.

DRAM, leaves only ownership metadata in CXL. StarfishOS partitions each system service into multiple owner instances instead of building one shared service for the whole pod because a single shared instance would push most service state into scarce cache-coherent CXL memory and make partial failures harder to contain: a machine crash during a service update could leave shared metadata in an inconsistent state that affects every client.

StarfishOS therefore keeps only ownership metadata in CXL and leaves service state inside per-machine DRAM. Each instance runs on one machine and owns a disjoint part of the global resource space, while the CXL metadata only records how to find that owner, e.g., mount-point-to-filesystem-instance mappings. Clients use this metadata to route service requests with local or cross-machine IPC, without sharing the instance’s full internal state.

The filesystem in Figure 8 is the concrete example. Each machine runs a filesystem manager and several filesystem instances, and each instance owns one mount point. On the first access to a path, the client asks the manager which instance owns the mount point and establishes an IPC connection; later requests go directly to that instance. If the owner is remote, the request uses cross-machine IPC. Other system services follow the same pattern. Supplementary material shows more details on filesystem service.

6.2 Partial Failure Recovery

StarfishOS aims to preserve system-service availability for surviving applications under partial machine failures, including network communication, complete filesystem access, and process management. Partial failures are handled by recovering only the lost service instances. Because system services are user-space processes, StarfishOS can micro-reboot a failed instance, i.e., restart just that service instance on a surviving machine without restarting the whole pod [46].

Whether recovery is needed depends on the service type. Uniform services, such as networking, provide the full interface at every instance, so clients can reconnect to any surviving instance after a failure. Process management is handled similarly in our design: new process or thread creation can be redirected to a surviving manager, while cleanup work for resources owned by the failed manager can be delayed or delegated. Non-uniform services that own disjoint resources, in contrast, require explicit instance recovery.

Table 2. Objects managed by StarfishOS.

Object	Description
CapGroup	capability array that references other objects
Thread	execution context and scheduling state
VMspace	virtual address-space metadata
PMO	physical memory pages
IPC	inter-process communication state
Notification	synchronization object

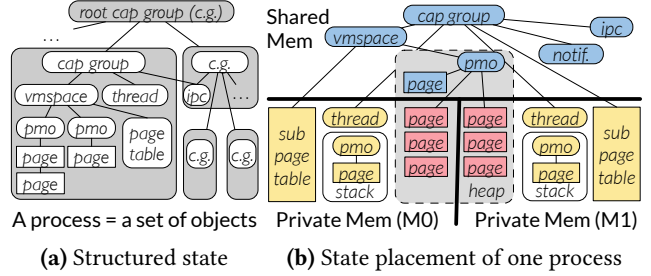


Figure 9. Application state partition.

In our current design, filesystem is the only non-uniform service that requires explicit instance recovery. After a machine failure, StarfishOS starts a replacement filesystem instance on another machine and rebuilds its state with Ananke-style p-log replay [46]. The p-log is stored in shared memory as recovery metadata rather than as runtime shared service state. It records each mutating filesystem operation before the service replies, so any update already reported as complete to the client is preserved even if it has not yet reached SSD. The replacement instance replays the p-log and then resumes serving the same mount point. Clients can then reconnect to the recovered instance and send requests.

7 Cross-machine Applications

With the kernel modules and collaborative system services in place, applications can already run transparently on a CXL pod. The remaining question is where their state should live. StarfishOS therefore partitions application state for efficiency and defines how applications recover from partial failures.

7.1 Application State Partition

State partitioning follows one principle: pay CXL latency only for state that must be shared across machines; keep all thread-local and machine-local state in DRAM.

Structured state organization. Capability-based micro-kernels [39, 54, 66, 69] represent system state as typed objects, and use capabilities to record which objects a process can reference and use. As shown in Figure 9(a), a process is therefore organized as a set of objects listed in Table 2. *CapGroup* stores the capabilities to these objects, making inter-object reachability explicit. This object-based organization lets StarfishOS track and place application state at object granularity.

Where to place each object? StarfishOS uses OS semantics to choose object placement. Figure 9(b) summarizes the resulting design along three cases:

(1) *Private state stays in DRAM.* Thread context and stacks are per-thread, so keeping them private avoids adding CXL latency. Each machine also keeps the local page-table state needed for its own execution path (further explained in §7.2).

(2) *Shared process metadata lives in CXL.* Process metadata like *CapGroup*, *VMSpace*, and *PMO* are placed in CXL because they require shared access yet are small and infrequently accessed. *IPC* and *Notification* state is also placed in CXL because these mechanisms are inherently cross-machine.

(3) *Heap pages are shared on demand.* The heap mixes local and cross-machine data, so placing the whole heap in CXL would waste bandwidth. StarfishOS therefore uses page-fault-driven migration to keep heap pages in DRAM by default and move only remotely accessed pages to CXL.

7.2 Page-fault-driven migration for Heap

Page-fault-driven migration enables transparent on-demand sharing of heap pages. A page is allocated in private DRAM of the machine that first touches it. When a remote access faults, StarfishOS migrates the page to CXL and makes it shared. Each machine keeps a sub page table, so a virtual page may be mapped on one machine and absent on others until sharing is required.

Figure 10 shows the migration path when machine0 accesses a page currently owned by machine1:

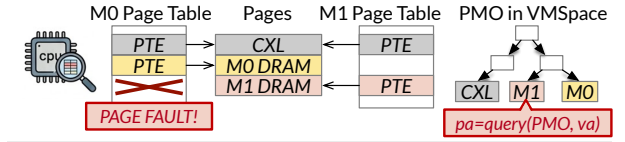
(1) Machine0 faults on a virtual address and queries *VMSpace* to determine whether the address has a valid mapping and, if so, whether the mapping points to a DRAM page. If so, machine0 identifies the owner machine and sends it a migration request through the IPC mechanism in §5.1.

(2) Machine1 handles the migration request. It allocates a CXL page and copies the page data from DRAM to CXL (①). It then atomically replaces the original PTE (page table entry) with a migration entry (②), a special illegal PTE [14, 15] that blocks concurrent accesses until the migration finishes. After flushing stale TLB entries (③), machine1 updates *VMSpace* (④) and installs a new PTE that points to the CXL page (⑤). Any access that encounters the migration entry waits for completion, then observes the new mapping and refills TLB.

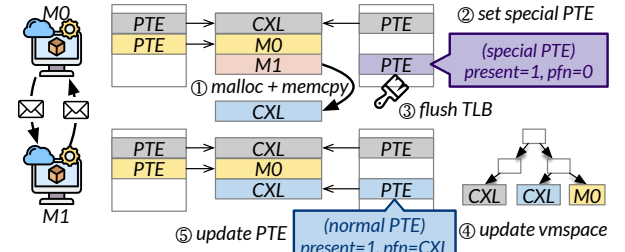
(3) After receiving the ACK together with the CXL page’s physical address, machine0 installs its local mapping to the CXL page and resumes execution.

StarfishOS supports concurrent page faults from different machines on the same page by maintaining a per-*VMSpace* list of in-flight migrations; a fault that finds an existing migration waits for it to finish. StarfishOS also bounds CXL usage by migrating pages back to DRAM and invalidating stale remote mappings when CXL is full. The current policy reclaims older migrated pages first and prefers migrating pages shared by fewer machines.

1. M0 page fault & Page exists and is on local DRAM



2. M0 Send Msg to ask M1 to migrate page



3. M0 receive ACK and map page

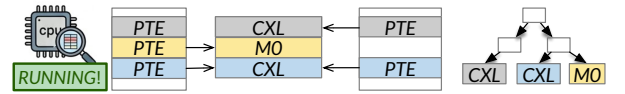


Figure 10. Transparent page placement across DRAM and CXL with Page-fault-driven migration.

7.3 Handling Partial Failure of Applications

StarfishOS does not hide partial failures from applications; it relies on their own recovery mechanisms. Transparent recovery would add common-case cost, and many applications already use write-ahead logging, restart, or replay from saved intermediate state.

StarfishOS instead focuses on keeping the OS usable after the failure. Surviving machines remain usable, files and flushed CXL state remain accessible, and applications can either restart from their own recovery data or continue with remaining threads if their state tolerates partial loss.

StarfishOS also keeps resource reclamation automatic. It traverses the capability tree to reclaim objects from exited or failed processes, and CXL-resident cap groups let the recycler find resources of cross-machine applications.

8 Evaluation

We evaluate StarfishOS using microbenchmarks and real applications to answer the following questions:

- How do the basic modules of StarfishOS perform? (§8.2)
- How does state partitioning contribute to efficiency? (§8.3)
- How can applications benefit from StarfishOS in auto-scaling and resource utilization use cases? (§8.4 and §8.5)
- How do applications recover from failures? (§8.6)

8.1 Experimental Setup

Testbed. Our evaluation platform has four Intel® Xeon® Gold 6418H CPUs (Sapphire Rapids), each with 24 physical cores and 64 GiB of DRAM. The server also includes a real 128 GiB CXL 1.1 memory device. CPU frequency is fixed at 2.1 GHz

Table 3. DRAM/CXL latency/bandwidth of our CXL 1.1 device and reported CXL 3.0 FPGA/ASIC results. Local and remote stand for local and remote NUMA. FPGA performance is based on real FPGA prototype [61], and ASIC performance is based on reports from IP providers [30]. Bandwidth: GB/s/channel for DRAM, GB/s/line for CXL.

Metric	Local DRAM	Remote DRAM	Local CXL	Remote CXL	FPGA CXL	ASIC CXL
Latency (ns)	107	160	241	383	456	170
Bandwidth (GB/s)	52	30	2	0.3	32	64

with Hyper-threading disabled, Intel® Turbo Boost disabled and NUMA balancing disabled.

Environment. Due to the lack of real CXL switch hardware, we emulate an 8-machine CXL pod by running 8 QE-MU/KVM instances, each with 12 physical CPU cores. Every two emulated machines share a NUMA node, and all machines access a CXL memory device. Except for applications running on StarfishOS, all other experiments are conducted on Linux v5.15 with Ubuntu 22.04. Among them, Tigon [37] is evaluated in Linux VMs to emulate CXL environment, while the other baselines run directly on bare metal without virtualization. Each experiment is performed 3 times following an initial warm-up run. VM exits account for 3–5% of execution time across the applications we evaluate.

CXL memory performance. As shown in Table 3, compared to CXL 3.0, our CXL 1.1 environment has comparable latency (246/384 ns v.s. 456 ns in FPGA and 170 ns in ASIC), while the bandwidth is substantially lower (2/0.3 GiB/s v.s. 32/64 GiB/s/line in FPGA and ASIC). Besides, our platform does not model the cacheline invalidation overhead in CXL 3.0 [61], which increases cache miss latency by 2–3× [61]. Nevertheless, these differences do not affect the conclusions of our evaluation: the lower bandwidth only puts our system at a disadvantage, and the latency is comparable; moreover, our partitioning strategy keeps most accesses on local DRAM and limits CXL accesses to a small fraction of the workload, thereby reducing the impact of the cacheline invalidation overhead in CXL 3.0 (for example, Matrix Multi. in our test has < 5% cache miss rate on accessing CXL memory, thus, the end-to-end runtime increases by at most 10%).

Workloads. We evaluate StarfishOS using microbenchmarks and the following real-world applications. All applications running on StarfishOS have no code changes.

- *Phoenix-2.0* [50]: single-machine MapReduce application.
- *DB×1000* [5, 68]: a single-node OLTP database system designed to make DBMS scalable on 1000-core processors.
- *GeminiGraph* [73]: a graph processing application with both single-machine and distributed modes.
- *CNN* [2]: a single-machine neural network inference.
- *Redis* [9], *Memcached* [7] and *LevelDB* [6]: memory-based and disk-based key-value stores.

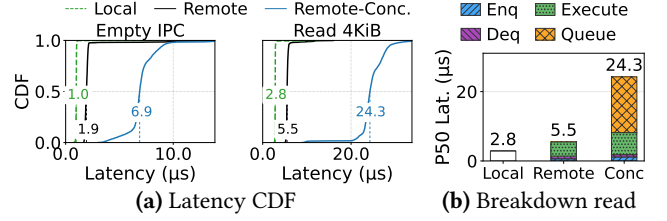


Figure 11. IPC latency CDF and breakdown.

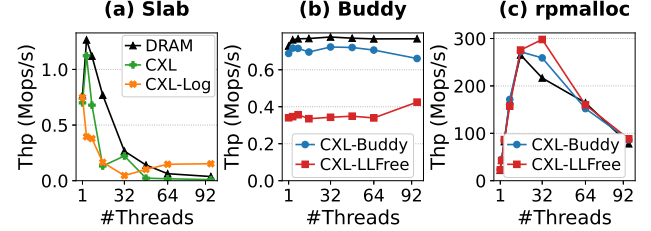


Figure 12. Memory allocator throughput.

8.2 Microbenchmarks

This section presents performance of basic kernel modules.

IPC. We evaluate local and remote IPC latency using two workloads: (1) *empty*, an IPC call with no server-side execution, and (2) *read 4 KiB*, which reads a 4 KiB file through IPC to filesystem. For remote IPC, we measure both the single-sender case (labeled as *Remote*) and four concurrent senders case (labeled as *Remote-Conc.*); local IPC is labeled as *Local*.

Figure 11(a) shows the latency CDF of local and remote IPC. For *empty*, P50 latency of local IPC is about 1.0 μs, while remote IPC is about 1.9 μs without concurrent senders and 6.9 μs with concurrent senders. For *read 4 KiB*, local IPC latency increases to 2.8 μs as the request includes executing filesystem read, and remote IPC latency increases to 5.5 μs and 24 μs without and with concurrent senders.

Figure 11(b) shows the latency breakdown of *read 4 KiB*; the *Local* column is a white bar marking P50 total only (component breakdown is given in text below), while *Remote* and concurrent senders use the stacked breakdown. For remote IPC, enqueue and dequeue on the IPC queue take about 1.4 μs in total, and handling the request takes about 2.0 μs. For remote IPC with concurrent senders, the extra 19 μs overhead is caused by queuing delay. For local read IPC (in total 2.8 μs), besides about 1.8 μs of executing the filesystem read, the time is spent on context switching (706 ns), IPC logic (200 ns), and kernel trap overhead (130 ns).

Scheduling/Notification. We evaluate scheduling/notification’s end-to-end latency. For scheduling, we start from the thread calling `sys_set_affinity` to when the thread is actually scheduled on target core. For notification, we start from a thread calling `sys_wait` to when a waiting thread is actually scheduled. We found that remote scheduling/notification adds negligible overhead compared to local scheduling/notification, both finish in about 100 μs. This is because the only overhead is enqueue/dequeue, which takes about 1–2 μs.

Memory Allocation. We evaluate three allocators: Slab, Buddy, and rpmalloc (our libc allocator). Slab and rpmalloc

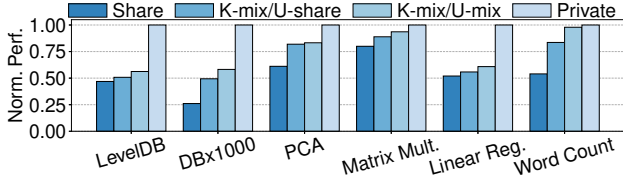


Figure 13. Performance across state-partition choices.

allocate 64–1024 B objects following the `rpmalloc` benchmark [11], while Buddy allocates 4 KiB and 2 MiB pages with equal probability. For each allocator, we compare a DRAM baseline with CXL-backed variants. The DRAM baseline uses the implementation in the base kernel. The CXL-backed variants use either the original buddy allocator (*CXL-Buddy*) or our LLFree-based buddy allocator (*CXL-LLFree*) [65]. For *Slab*, we evaluate the effect of undo logging within the *CXL-LLFree* setting by comparing *CXL* (without undo logging) and *CXL-Log* (with undo logging).

Figure 12 shows that kernel-space allocation is noticeably more sensitive to CXL placement than user-space allocation. In contrast, the impact on user-space `rpmalloc` is smaller, because most `malloc/free` operations are served from thread-local state (e.g., free lists), and therefore touch cached local metadata more often than the kernel allocator. Also, our usage of *Undo-log* mechanism and *CXL-LLFree* decreases kernel-space allocator’s performance due to cacheline flush, but have minimal impact on user-space `rpmalloc`.

8.3 Effect of State Partitioning

We evaluate the effectiveness of StarfishOS’s state partitioning by running several applications across two machines with four state-partition configurations: (1) *Share*: place all kernel and application state in CXL; (2) *K-mix/U-share*: partition only kernel state, while all application state in CXL; (3) *K-mix/U-mix*: partition both application and kernel state; (4) *Private*: all state in private DRAM (ideal case).

Figure 13 shows application performance (LevelDB and DBx1000’s throughput; others’ 1/execution time) normalized to the *Private* setup across four state-partition choices. Results show that StarfishOS’s state partitioning (*K-mix/U-mix*) achieves 1.2–2.2× performance improvement compared to placing all state in shared memory (*Share*), and 1.0–1.7× improvement over only partitioning kernel state (*K-mix/U-share*). These results demonstrate the benefit of state partitioning. CPU-intensive applications (e.g., PCA, Matrix Multi.) achieve similar performance across all setups because most data can be cached in CPU cache.

8.4 Auto-scaling Applications

This case study evaluates whether StarfishOS can auto-scale existing single-machine applications, and outperform their distributed and CXL-optimized baselines.

Configs. We use three unmodified multithreaded applications: (1) Matrix Multi. in Phoenix [50] (multiply one

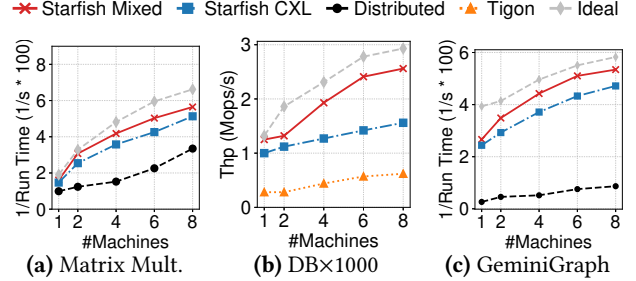


Figure 14. Auto-scaling single-machine applications.

Table 4. Mem usage in auto-scaling apps (8 machines). We fix DBx1000 and Tigon [37]’s CXL usage to 500 MiB.

Mem	Matrix Mult.	DBx1000	Tigon	GeminiGraph
Total	1 GiB	26 GiB	26 GiB	9 GiB
CXL	109 MiB	500 MiB	500 MiB	600 MiB

4000×4000 matrix); (2) DBx1000 [5, 68] running TPCC [12] with 64 warehouses, 50% New-order, 50% Payment, 15% cross-warehouse transactions; and (3) GeminiGraph [73] running PageRank [49] on twitter-2010 [41].

We test two StarfishOS configurations: *StarfishOS-CXL*, which places all heap and page table in CXL, and *StarfishOS-Mixed*, which places only shared part of the heap in CXL.

We compare StarfishOS with distributed baselines (*Distributed*) for all applications, and compare DBx1000 running on StarfishOS with Tigon [37]. For Phoenix, we compare with a multi-process local TCP-based baseline. For DBx1000, we compare with Tigon [37]’s open-source implementation with WAL disabled for fairness. For GeminiGraph, which provides both single-machine and distributed modes, we compare with its native distributed MPI implementation. For reference, we also report each application’s ideal performance (labeled as *Ideal*), measured under multithreaded execution on Linux with all state in DRAM.

Results. Figure 14 shows that applications can scale from 1 to 8 machines with 3.5× (Matrix Mult.), 2.1× (DBx1000), and 2.0× (GeminiGraph) speedup under *StarfishOS-Mixed*. Across all three applications, *StarfishOS-Mixed* consistently outperforms *StarfishOS-CXL*, indicating the effectiveness of partitioning heap state. Besides, *StarfishOS-Mixed* achieves 85%–92% of the ideal performance at 8 machine, while only consumes several hundred MiB of CXL memory, compared with total working set size of 1–26 GiB, as shown in Table 4.

StarfishOS v.s. distributed baselines. Applications running on StarfishOS achieve 5.2× (GeminiGraph) and 1.7× (Matrix Mult.) speedup compared to their distributed baselines. The performance gain of StarfishOS comes from replacing network-based distributed communication with direct load/store accesses over CXL. For Phoenix, StarfishOS avoids the memory-copy and network-transfer overhead of MapReduce shuffle. For DBx1000, StarfishOS avoids the cost of network-based 2PC by enabling direct access to shared state. For GeminiGraph, StarfishOS eliminates the message passing

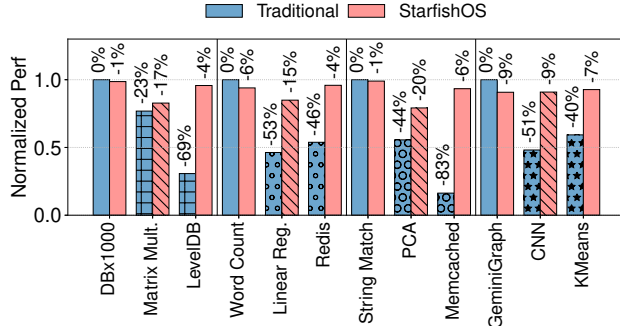


Figure 15. Application performance on Traditional OSes and StarfishOS.

between graph partitions in the distributed design, allowing partitions to directly update remote vertices through load/store accesses over CXL.

DBx1000 on StarfishOS v.s. Tigon. StarfishOS achieves 4.1× the throughput of Tigon at 8 machines, suggesting that transparent placement already captures much of the benefit of CXL-based shared-state access. This result is notable because Tigon is an application-specific redesign that uses application semantics to place cross-machine shared data in CXL on demand.¹

8.5 CPU Resource Utilization

StarfishOS enables applications to utilize stranded CPU resources by running applications across multiple machines. In Figure 15, we evaluate 12 datacenter applications on traditional OSes (our base microkernel) and StarfishOS. Each application has 8 threads, and we have 8 machines, each with 12 cores. Under traditional OSes, each application is restricted to a single machine. Taking three applications and two machines as a group, one application runs exclusively on one machine (non-hatched bars in *Traditional*), while the other two applications are co-located on the other machine (hatched bars in *Traditional*). Bars with the same hatch pattern indicate applications co-located on the same machine. In contrast, with StarfishOS, applications can span multiple machines without requiring any code modifications. Consequently, in each group, two applications run independently on separate machines (unhatched bars in *StarfishOS*), while the remaining application executes across both machines (hatched bars in *StarfishOS*). All results in Figure 15 are normalized to the performance of exclusive execution of 8-threaded applications on 8 cores on our base microkernel.

The results demonstrate that StarfishOS’s SSI capability can effectively utilize stranded CPU resources scattered across the machines. As shown in Figure 15, co-located applications on a traditional OS (traditional-hatched) achieve only 17%–77% of the performance of exclusive execution. In contrast, applications that run across machines on StarfishOS

¹We note that this is not a strictly apples-to-apples comparison, since Tigon and DBx1000 are based on different codebases. In our test, Tigon is bottlenecked by limited intra-machine scalability.

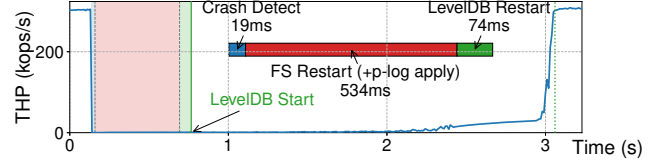


Figure 16. Recovery of LevelDB on StarfishOS.

(StarfishOS-hatched) achieve 83%–93% of exclusive performance, while other eight applications (StarfishOS-unhatched) are nearly unaffected. These results highlight the benefit of StarfishOS’s ability to utilize all stranded CPU resources with acceptable performance downgrades. We further evaluated several alternative grouping and co-location strategies, all of which exhibit similar performance gains with StarfishOS, reinforcing the generality of these benefits.

8.6 Partial Failure Resilience

To evaluate how applications recover from partial failures of machines, we deploy LevelDB on Machine-A and crash Machine-A. Machine failure is simulated by killing the QEMU instance. After 19 ms, the failure detector detects the failure of Machine-A, and micro-reboots an empty filesystem instance on another machine in 5.5 ms. The empty filesystem instance recovers the state gap with the crashed instance following Ananke-style replay [46]. Specifically, it replays the p-log in 528 ms (to replay 446 entries after last LevelDB’s flush). LevelDB can then reconstruct its in-memory state in 73 ms and warm up its execution in 3 s.

9 Related Work

StarfishOS builds on several lines of prior work, but combines them in a different setting. Our cross-machine kernel modules and collaborative services build on decades of SSI design [19, 26, 45, 58, 63]. Our state partitioning draws inspiration from SASOS [28, 35, 43], Corey [23], and Mitosis [16], which highlight the benefits of explicit state management. For partial-failure resilience, StarfishOS adopts lock-free, detectably recoverable data structures [18, 31] and lock-free allocators [65] motivated by recent works on CXL partial-failure resilience [62, 70, 74]. Our page-fault-triggered migration and replicated page-table design are also inspired by prior NUMA and page-table replication work [16]. Our contribution is integrating these ideas into an OS-level SSI that transparently runs unmodified applications on a CXL pod while preserving availability under partial failures.

10 Conclusion

StarfishOS is the first OS-level SSI to use CXL’s shared-memory primitive for efficient cross-machine synchronization. Built on a state-partitioned microkernel, it keeps CXL-resident state small, provides OS-level partial-failure resilience, and lets single-machine applications scale transparently across multiple machines with substantial speedups.

References

- [1] 2019. How to and how not to use seL4 IPC. <https://microkerneldude.org/2019/03/07/how-to-and-how-not-to-use-sel4-ipc/>.
- [2] 2025. CNN. <https://github.com/ekut-es/pico-cnn>.
- [3] 2025. Compute Express Link (CXL). <https://www.computeexpresslink.org/>.
- [4] 2025. CXL Consortium. Compute Express Link Specification, Revision 3.1, 2025. <https://computeexpresslink.org/wp-content/uploads/2025/02/CXL-3.1-Specification.pdf>.
- [5] 2025. DBx1000. <https://github.com/yxymit/DBx1000>.
- [6] 2025. LevelDB. <https://dbdb.io/db/leveldb>.
- [7] 2025. Memcached - a distributed memory object caching system. <https://memcached.org/>.
- [8] 2025. Musl Libc. <https://musl.libc.org/>.
- [9] 2025. Redis. <https://redis.io/>.
- [10] 2025. RocksDB Home Page. <https://www.rocksdb.org>.
- [11] 2025. Rpmalloc Benchmark. <https://github.com/mjansson/rpmalloc-benchmark>.
- [12] 2025. TPC Benchmark C. <https://www.tpc.org/tpcc/>.
- [13] 2025. TreesLS. <https://ipads.se.sjtu.edu.cn/projects/treesls.html>.
- [14] 2026. Page migration. https://docs.kernel.org/mm/page_migration.html.
- [15] 2026. Process Addresses. https://docs.kernel.org/mm/process_addr.html.
- [16] Reto Achermann, Ashish Panwar, Abhishek Bhattacharjee, Timothy Roscoe, and Jayneel Gandhi. 2020. Mitosis: Transparently Self-Replicating Page-Tables for Large-Memory Machines. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '20). Association for Computing Machinery, New York, NY, USA, 283–300. <https://doi.org/10.1145/3373376.3378468>
- [17] C. Amza, A.L. Cox, S. Dwarkadas, P. Keleher, Honghui Lu, R. Rajamony, Weimin Yu, and W. Zwaenepoel. [n. d.]. TreadMarks: Shared Memory Computing on Networks of Workstations. 29, 2 ([n. d.]), 18–28. <https://doi.org/10.1109/2.485843>
- [18] Hagit Attiya, Ohad Ben-Baruch, Panagiota Fatourou, Danny Hendler, and Eleftherios Kosmas. 2022. Detectable recovery of lock-free data structures. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Seoul, Republic of Korea) (PPoPP '22). Association for Computing Machinery, New York, NY, USA, 262–277. <https://doi.org/10.1145/3503221.3508444>
- [19] Antonio Barbalace, Marina Sadini, Saif Ansary, Christopher Jelesnianski, Akshay Ravichandran, Cagil Kendir, Alastair Murray, and Binoy Ravindran. 2015. Popcorn: bridging the programmability gap in heterogeneous-ISA platforms. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) (EuroSys '15). Association for Computing Machinery, New York, NY, USA, Article 29, 16 pages. <https://doi.org/10.1145/2741948.2741962>
- [20] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhania. 2009. The multikernel: a new OS architecture for scalable multicore systems. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (SOSP '09). Association for Computing Machinery, New York, NY, USA, 29–44. <https://doi.org/10.1145/1629575.1629579>
- [21] B. Bershad, T. Anderson, E. Lazowska, and H. Levy. 1989. Lightweight remote procedure call. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles* (SOSP '89). Association for Computing Machinery, New York, NY, USA, 102–113. <https://doi.org/10.1145/74850.74861>
- [22] Jeff Bonwick. 1994. The slab allocator: an object-caching kernel memory allocator (USTC'94). USENIX Association, USA, 6.
- [23] Silas Boyd-Wickizer, Haibo Chen, Rong Chen, Yandong Mao, Frans Kaashoek, Robert Morris, Aleksey Pesterev, Lex Stein, Ming Wu, Yuehua Dai, Yang Zhang, and Zheng Zhang. 2008. Corey: an operating system for many cores. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) (OSDI '08). USENIX Association, USA, 43–57.
- [24] Qingchao Cai, Wentian Guo, Hao Zhang, Divyakant Agrawal, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, Yong Meng Teo, and Sheng Wang. [n. d.]. Efficient Distributed Memory Management with RDMA and Caching. 11, 11 ([n. d.]), 1604–1617. <https://doi.org/10.14778/3236187.3236209>
- [25] George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman, and Armando Fox. 2004. Microreboot—A Technique for Cheap Recovery. In *6th Symposium on Operating Systems Design and Implementation* (OSDI '04). USENIX Association, San Francisco, CA.
- [26] J. Chapin, M. Rosenblum, S. Devine, T. Lahiri, D. Teodosiu, and A. Gupta. 1995. Hive: fault containment for shared-memory multiprocessors. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles* (Copper Mountain, Colorado, USA) (SOSP '95). Association for Computing Machinery, New York, NY, USA, 12–25. <https://doi.org/10.1145/224056.224059>
- [27] Matthew Chapman and Gernot Heiser. 2005. Implementing Transparent Shared Memory on Clusters Using Virtual Machines. In *2005 USENIX Annual Technical Conference* (USENIX ATC 05). USENIX Association, Anaheim, CA. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/implementing-transparent-shared-memory-clusters>
- [28] Jeffrey S. Chase, Henry M. Levy, Michael J. Feeley, and Edward D. Lazowska. 1994. Sharing and Protection in a Single-Address-Space Operating System. *ACM Trans. Comput. Syst.* 12, 4 (November 1994), 271–307.
- [29] David Cheriton. 1988. The V distributed system. *Commun. ACM* 31, 3 (mar 1988), 314–333. <https://doi.org/10.1145/42392.42400>
- [30] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Comput. Surv.* 56, 11, Article 290 (July 2024), 37 pages. <https://doi.org/10.1145/3669900>
- [31] Michal Friedman, Maurice Herlihy, Virendra Marathe, and Erez Petrank. 2018. A persistent lock-free queue for non-volatile memory. (2018), 28–40. <https://doi.org/10.1145/3178487.3178490>
- [32] Jim Gray. 1978. *Notes on Data Base Operating Systems*. Technical Report. Berlin, Heidelberg, 393–481 pages.
- [33] Jinyu Gu, Xinyue Wu, Wentai Li, Nian Liu, Zeyu Mi, Yubin Xia, and Haibo Chen. 2020. *Harmonizing Performance and Isolation in Microkernels with Efficient Intra-Kernel Isolation and Communication*. USENIX Association.
- [34] Gernot Heiser and Kevin Elphinstone. 2016. L4 Microkernels: The Lessons from 20 Years of Research and Deployment. *ACM Trans. Comput. Syst.* 34, 1, Article 1 (apr 2016), 29 pages. <https://doi.org/10.1145/2893177>
- [35] Gernot Heiser, Kevin Elphinstone, Stephen Russell, and Jerry Vochtelloo. 1994. Mungi: A Distributed Single-Address-Space Operating System. In *Proceedings of the 17th Australasian Computer Science Conference* (ACSC). Christchurch, New Zealand, 271–80.
- [36] Jorrit N. Herder, Herbert Bos, Ben Gras, Philip Homburg, and Andrew S. Tanenbaum. 2007. Failure Resilience for Device Drivers. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks* (DSN'07). IEEE, 41–50.
- [37] Yibo Huang, Haowei Chen, Newton Ni, Yan Sun, Vijay Chidambaram, Dixing Tang, and Emmett Witchel. 2025. Tigon: a distributed database for a CXL pod. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (Boston, MA, USA) (OSDI '25). USENIX Association, USA, Article 7, 20 pages.

- [38] Sunita Jain, Nagaradhes Yelleswarapu, Hasan Al Maruf, and Rita Gupta. 2024. Memory Sharing with CXL: Hardware and Software Design Approaches. arXiv:2404.03245 [cs.ET] <https://arxiv.org/abs/2404.03245>
- [39] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. SeL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (SOSP '09). Association for Computing Machinery, New York, NY, USA, 207–220.
- [40] Kenneth C. Knowlton. 1965. A fast storage allocator. *Commun. ACM* 8, 10 (oct 1965), 623–624. <https://doi.org/10.1145/365628.365655>
- [41] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media?. In *Proceedings of the 19th International Conference on World Wide Web* (Raleigh, North Carolina, USA) (WWW '10). Association for Computing Machinery, New York, NY, USA, 591–600. <https://doi.org/10.1145/1772690.1772751>
- [42] Seung-seob Lee, Yanpeng Yu, Yupeng Tang, Anurag Khandelwal, Lin Zhong, and Abhishek Bhattacharjee. [n. d.]. MIND: In-Network Memory Management for Disaggregated Data Centers. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (New York, NY, USA, 2021-10-26). Association for Computing Machinery, 488–504. <https://doi.org/10.1145/3477132.3483561>
- [43] Ian Leslie, Derek McAuley, Richard Black, Timothy Roscoe, Paul Barham, David Evers, Robin Fairbairns, and Eoin Hyden. 1996. The Design and Implementation of an Operating System to Support Distributed Multimedia Applications. *IEEE Journal on Selected Areas in Communications* 14 (1996), 1280–1297.
- [44] Kai Li. 1988. IVY: A Shared Virtual Memory System for Parallel Computing. In *International Conference on Parallel Processing*. <https://api.semanticscholar.org/CorpusID:29219727>
- [45] Felix Xiaozhu Lin, Zhen Wang, and Lin Zhong. 2014. K2: a mobile operating system for heterogeneous coherence domains. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems* (Salt Lake City, Utah, USA) (ASPLOS '14). Association for Computing Machinery, New York, NY, USA, 285–300. <https://doi.org/10.1145/2541940.2541975>
- [46] Jing Liu, Yifan Dai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2025. Fast, Transparent Filesystem Microkernel Recovery with Ananke. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 1–18. <https://www.usenix.org/conference/fast25/presentation/liu-jing>
- [47] Arm Ltd. 2024. Arm OCP CMS – Design Considerations in Adding Coherency to CXL Devices (HDM-DB). OCP-CMS Workgroup Presentation. [https://ocp-all.groups.io/g/OCP-CMS/attachment/222/1/Arm%20OCP%20CMS%20-%20Design%20Considerations%20in%20Adding%20Coherency%20to%20CXL%20Devices%20\(HDM-DB\)%20-%20Jan%2012%202024.pdf](https://ocp-all.groups.io/g/OCP-CMS/attachment/222/1/Arm%20OCP%20CMS%20-%20Design%20Considerations%20in%20Adding%20Coherency%20to%20CXL%20Devices%20(HDM-DB)%20-%20Jan%2012%202024.pdf) OCP-CMS meeting, Jan. 12, 2024.
- [48] Jacob Nelson, Brandon Holt, Brandon Myers, Preston Briggs, Luis Ceze, Simon Kahan, and Mark Oskin. 2015. Latency-Tolerant Software Distributed Shared Memory. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*. USENIX Association, Santa Clara, CA, 291–305. <https://www.usenix.org/conference/atc15/technical-session/presentation/nelson>
- [49] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The PageRank Citation Ranking : Bringing Order to the Web. In *The Web Conference*. <https://api.semanticscholar.org/CorpusID:1508503>
- [50] Colby Ranger, Ramanan Raghuraman, Arun Penmetsa, Gary Bradski, and Christos Kozyrakis. 2007. Evaluating MapReduce for Multi-core and Multiprocessor Systems. 2007 IEEE 13th International Symposium on High Performance Computer Architecture, 13–24.
- [51] Andy Rudoff, Chet Douglas, and Tiffany Kasanicky. 2021. Persistent Memory in CXL. <https://www.snia.org/sites/default/files/PMSummit/2021/snia-pm-cs-summit-Rudoff-PM-in-CXL-2021.pdf> SNIA Persistent Memory and Computational Storage Summit.
- [52] Dirk Schmidl, Christian Terboven, Andreas Wolf, Dieter an Mey, and Christian Bischof. 2010. How to Scale Nested OpenMP Applications on the ScaleMP vSMP Architecture. In *2010 IEEE International Conference on Cluster Computing*. 29–37. <https://doi.org/10.1109/CLUSTER.2010.38>
- [53] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. 2018. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 69–87. <https://www.usenix.org/conference/osdi18/presentation/shan>
- [54] Jonathan S. Shapiro, Jonathan M. Smith, and David J. Farber. 1999. EROS: A Fast Capability System. In *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles* (Charleston, South Carolina, USA) (SOSP '99). Association for Computing Machinery, 170–185.
- [55] Anton Shilov. 2025. Samsung, MemVerge, and H3 build 2TB CXL Memory Pool. <https://www.anandtech.com/show/20009/samsung-memverge-and-h3-build-2tb-cxl-memory-pool>
- [56] Xiang Song, Haibo Chen, Rong Chen, Yuanxuan Wang, and Binyu Zang. 2011. A case for scaling applications to many-core with OS clustering. In *Proceedings of the Sixth Conference on Computer Systems* (Salzburg, Austria) (EuroSys '11). Association for Computing Machinery, New York, NY, USA, 61–76. <https://doi.org/10.1145/1966445.1966452>
- [57] Michael M. Swift, Muthukaruppan Annamalai, Brian N. Bershad, and Henry M. Levy. 2006. Recovering device drivers. *ACM Transactions on Computer Systems* 24, 4 (2006), 333–360.
- [58] Bruce Walker, Gerald Popek, Robert English, Charles Kline, and Greg Thiel. 1983. The LOCUS distributed operating system. In *Proceedings of the Ninth ACM Symposium on Operating Systems Principles* (Bretton Woods, New Hampshire, USA) (SOSP '83). Association for Computing Machinery, New York, NY, USA, 49–70. <https://doi.org/10.1145/800217.806615>
- [59] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. 2021. Concordia: Distributed Shared Memory with In-Network Cache Coherence. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 277–292. <https://www.usenix.org/conference/fast21/presentation/wang>
- [60] Zhao Wang, Yiqi Chen, Cong Li, Yijin Guan, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. CTXNL: A Software-Hardware Co-designed Solution for Efficient CXL-Based Transaction Processing. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 192–209. <https://doi.org/10.1145/3676641.3716244>
- [61] Zhao Wang, Yiqi Chen, Cong Li, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. Enabling Efficient Transaction Processing on CXL-Based Memory Sharing. In *the ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [62] Xingda Wei, Haotian Wang, Tianxia Wang, Rong Chen, Jinyu Gu, Pengfei Zuo, and Haibo Chen. 2023. Transactional Indexes on (RDMA or CXL-based) Disaggregated Memory with Repairable Transaction. arXiv:2308.02501 [cs.DB] <https://arxiv.org/abs/2308.02501>
- [63] David Wentzlaff, Charles Gruenwald, Nathan Beckmann, Kevin Modzelewski, Adam Belay, Lamia Youseff, Jason Miller, and Anant Agarwal. 2010. An operating system for multicore and clouds: mechanisms and implementation (SoCC '10). Association for Computing Machinery, New York, NY, USA, 3–14. <https://doi.org/10.1145/1807128>

- [64] Emmett Witchel. 2024. Challenges and Opportunities for Systems Using CXL Memory. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 2. <https://doi.org/10.1145/3620666.3655590>
- [65] Lars Wrenger, Florian Rommel, Alexander Halbuer, Christian Dietrich, and Daniel Lohmann. 2023. LLFree: Scalable and Optionally-Persistent Page-Frame Allocation. In *2023 USENIX Annual Technical Conference (USENIX ATC '23)*. USENIX Association, Boston, MA, 897–914. <https://www.usenix.org/conference/atc23/presentation/wrenger>
- [66] Fangnuo Wu, Mingkai Dong, Gequan Mo, and Haibo Chen. 2023. TreeSLS: A Whole-system Persistent Microkernel with Tree-structured State Checkpoint on NVM. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 1–16. <https://doi.org/10.1145/3600006.3613160>
- [67] Yi Xu, Suyash Mahar, Ziheng Liu, Mingyao Shen, and Steven Swanson. 2024. CXL Shared Memory Programming: Barely Distributed and Almost Persistent. arXiv:2405.19626 [cs.DC] <https://arxiv.org/abs/2405.19626>
- [68] Xiangyao Yu, George Bezerra, Andrew Pavlo, Srinivas Devadas, and Michael Stonebraker. 2014. Staring into the abyss: an evaluation of concurrency control with one thousand cores. *Proc. VLDB Endow.* 8, 3 (nov 2014), 209–220. <https://doi.org/10.14778/2735508.2735511>
- [69] Gerd Zellweger, Simon Gerber, Kornilios Kourtis, and Timothy Roscoe. 2014. Decoupling Cores, Kernels, and Operating Systems. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation* (Broomfield, CO) (OSDI '14). USENIX Association, USA, 17–31.
- [70] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 658–674. <https://doi.org/10.1145/3600006.3613135>
- [71] Xu Zhang, Ke Liu, Yuan Hui, Xiaolong Zheng, Yisong Chang, Yizhou Shan, Guanghui Zhang, Ke Zhang, Yungang Bao, Mingyu Chen, and Chenxi Wang. 2025. DRack: a CXL-disaggregated rack architecture to boost inter-rack communication. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) (USENIX ATC '25). USENIX Association, USA, Article 74, 19 pages.
- [72] Yuhong Zhong, Daniel S. Berger, Pantea Zardoshti, Enrique Saurez, Jacob Nelson, Antonis Psistakis, Joshua Fried, and Asaf Cidon. 2025. My CXL Pool Obviates Your PCIe Switch. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems* (Banff, AB, Canada) (HotOS '25). Association for Computing Machinery, New York, NY, USA, 58–66. <https://doi.org/10.1145/3713082.3730393>
- [73] Xiaowei Zhu, Wenguang Chen, Weimin Zheng, and Xiaosong Ma. 2016. Gemini: A Computation-Centric Distributed Graph Processing System. In *12th USENIX Symposium on Operating Systems Design and Implementation* (OSDI 16). USENIX Association, Savannah, GA, 301–316. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/zhu>
- [74] Zhiting Zhu, Newton Ni, Yibo Huang, Yan Sun, Zhipeng Jia, Nam Sung Kim, and Emmett Witchel. 2024. Lupin: Tolerating Partial Failures in a CXL Pod. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems* (Austin, TX, USA) (DIMES '24). Association for Computing Machinery, New York, NY, USA, 41–50. <https://doi.org/10.1145/3698783.3699377>